Tom **Olsen**

PhD in Computer Science



Personal

Tom Olsen, 1992
Comp. Science PhD
61440 Oberursel
Tom-Olsen
Tom Olsen

Areas of specialization

Modern C++ & software architecture
GPU & parallel computing
Real-time rendering
Scientific computing
Numerical methods
Physics simulation
Fluid & radiation transport
Computer vision & imaging
Data visualization

Interests

Rock climbing
Mountains
Games
Physics
Math

About me

I am a physicist and computer scientist specializing in performance-oriented C++ software at the intersection of GPU computing, numerical simulation, and real-time graphics. My doctoral research focuses on developing numerical methods for radiation transport in curved spacetime for use in general-relativistic magnetohydrodynamics simulations. In my spare time, I develop Ember, my custom Vulkan game engine that allows me to explore real-time rendering, engine architecture, and performance-oriented programming. I am a quick learner with a proven ability to adapt swiftly to new technologies, programming languages, and mathematical concepts. Additionally, I am a team player and want to apply my skills in a new working environment. My ambitions are to work in a challenging and dynamic environment where I can contribute to the development of innovative solutions and broaden my knowledge in the fields of computer science, physics, and mathematics.

Experience

Aug. 2024 - Today

Game Engine - Ember

· Designed and developed Ember, a cross-platform C++20 engine for real-time graphics and GPU-accelerated simulation using Vulkan, SDL3, Dear ImGui, and HLSL.
· Architected a backend-independent core with explicit interfaces and interchangeable renderer, window, GUI, and null backends for game, editor, debug, and headless applications.
· Engineered a Vulkan render graph spanning deferred and forward rendering, shadows, post-processing, and compute passes, with explicit cross-queue synchronization and asynchronous, triple-buffered GPU workflows.
· Built an HLSL-to-SPIR-V shader pipeline with reflection-driven descriptor layouts and automatic resource binding, complemented by VMA-backed allocation and deferred resource destruction.
· Developed an entity-component system, multithreaded task execution and profiling, editor tooling, diagnostics, and automated unit and integration tests.

Jun. 2025 - Today

Digital Film Technology

· Responsible for the film scanners computer vision image stabilization as part of an R&D software engineering team.
· Designed, trained, and evaluated convolutional neural networks (AI) for image artifact detection and removal using PyTorch.
· Integrated in-house and third-party AI models into a C++ framework using TensorRT for production deployment.
· Collaborated closely with other developers and researchers in an agile workflow (Jira), contributing to sprint planning, task breakdown, and cross-component integration.
· Actively participated in peer code reviews, providing and incorporating feedback to maintain code quality, performance, and long-term maintainability.
· Designed and led a pilot initiative introducing agentic coding workflows with Claude and Codex to the R&D team.

Jul. 2020 - Aug. 2024

Computer Science PhD

· Combined multiple concepts of physics, mathematics, and computer science to develop novel ideas to solve complex problems.
· Implemented abstract physics and mathematics into high-performance multithreaded C++ code.
· Developed 3D visualization library in Unity to plot complex datastructures with C# and HLSL.

Okt. 2023 - Feb. 2024

Lead Tutor

· Organized and led a team of fellow PhD tutors.
· Coordinated schedules and delegated tasks.
· Ensured the delivery of high-quality tutoring sessions.

Aug. 2016 - Aug. 2022

Tutoring positions

· Cooperation and teamwork with fellow tutors.
· Taught academic level of mathematics, physics, and computer science to hundreds of students over the course of several years.
· Guided students in problem-solving techniques, critical thinking, and analytical reasoning.
· Experienced in delivering presentations and speeches to large audiences (more than 100 listeners), effectively communicating complex ideas in a clear and engaging manner.

Tom **Olsen**

PhD in Computer Science



Personal

Tom Olsen, 1992
Comp. Science PhD
61440 Oberursel
Tom-Olsen
Tom Olsen

Areas of specialization

Modern C++ & software architecture
GPU & parallel computing
Real-time rendering
Scientific computing
Numerical methods
Physics simulation
Fluid & radiation transport
Computer vision & imaging
Data visualization

Interests

Rock climbing
Mountains
Games
Physics
Math

Education

Aug. 2024	**Computer Science PhD** · magna cum laude Goethe University · Frankfurt am Main "General Relativistic Lattice Boltzmann Method for Radiation Transport"
Feb. 2020	**Master of Physics** · 1.0 Goethe University · Frankfurt am Main "Solving the Klein-Gordon equation in static curved spacetimes on GPUs"
Nov. 2017	**Bachelor of Physics** · 1.6 Goethe University · Frankfurt am Main "Error studies for the MYRRHA injector"
Jul. 2013	**Abitur** · 1.6 Gymnasium Schwertstraße · Solingen
May 2010	**High school diploma** · Honorary Belle Fourche High School· South Dakota, USA

Programming

C++
C#
Python
HLSL
Vulkan
CUDA
OpenCV

Software

CMake
Git
Unity
Blender
LaTeX

Languages

German
English
Turkish

Workshops

2023	Threee day workshop on **Leadership and Career Development**.
2021	Threee day workshop on **Leading Teams in a Research Environment**.
2021	Threee day workshop on **Making an Impact as an Effective Researcher**.

Publications

2024	"General Relativistic Lattice Boltzmann Method for Radiation Transport", in: Monthly Notices of the Royal Astronomical Society.

Projects

Ongoing	Ember, a cross-platform C++20 engine for real-time graphics and GPU-accelerated simulation, designed for game, editor, debug, and headless applications.
Ongoing	Self-managed home lab server for personal services, storage, and backups; a practical environment for Linux administration, networking, containers, and automation.
2024	2D and 3D plotting library in Unity, leveraging GPU instancing, custom vertex, fragment, surface, and compute shaders.